DATA SCIENCE CAPSTONE PROJECT REPORT

SpaceX Falcon 9 First-Stage Landing Prediction

Applied Data Science Capstone • IBM

Data collection | Wrangling | EDA | SQL | Visualization | Machine learning

SUBMISSION NOTE: Add your GitHub repository URL and replace illustrative charts/metrics with outputs generated from your own completed notebook before final submission.

Executive Summary

Business question

Can public launch and payload data help estimate whether
a Falcon 9 first stage will land successfully?

Methods

API + web data collection; cleaning and feature engineering;
EDA, SQL queries, Folium, Plotly Dash and classification models.

Decision value

Support launch-cost analysis and mission planning by
identifying factors associated with recovery success.

Key takeaway

- Use launch site, payload mass, orbit, booster/version and mission timing as candidate predictors.
- Compare multiple classifiers using held-out evaluation and a confusion matrix.
- Treat patterns as associations, not proof of causation; verify every result from the executed notebook.

Integrity note: numerical chart values in this draft are illustrative placeholders, not verified project outputs.

Introduction & Problem Definition

Background

- Reusable rocket first stages can reduce launch costs when recovery and refurbishment are successful.
- SpaceX launch records provide mission, vehicle, payload, site and outcome information for analysis.

Problem statement

Build a reproducible data-science workflow that explores historical Falcon 9 launches and evaluates whether launch context can predict first-stage landing success.

Analytical target

Binary outcome: first-stage landing successful vs. unsuccessful.

Users

Analysts, mission planners and teams studying launch economics.

Scope

Historical data; model estimates are not operational guarantees.

Data Collection — SpaceX API

1. Request

Call SpaceX API endpoints for launch records and related entities.

2. Parse

Read JSON responses; extract launch date, site, rocket, payload and outcomes.

3. Join

Connect launch records to rocket, payload, launchpad and core details.

4. Save

Store raw responses and curated tables/CSV files for reproducibility.

Record in the repository

- Endpoint(s) used and request parameters
- Fields selected and JSON-to-table transformation
- Missing-data handling, API date/version and collection script

GitHub URL: ADD YOUR REPOSITORY URL HERE

Data Collection — Web Scraping

1 • Inspect page

Identify launch table, columns, pagination and source terms of use.

2 • Extract

Use requests/BeautifulSoup or pandas read_html where permitted.

3 • Validate

Normalize dates and labels; remove duplicates;
compare records with API data.

Good practice

- Respect robots.txt, rate limits and website terms.
- Keep source URL, retrieval date and parsing assumptions.
- Avoid scraping when an official API provides the same reliable field.

GitHub URL: ADD YOUR REPOSITORY URL HERE

Data Wrangling Methodology

Quality checks

Inspect schema, duplicates, nulls and inconsistent labels.

Feature engineering

Create target landing outcome; parse dates; derive year and payload mass.

Data integration

Join launch, payload, rocket/core and launch-site tables using stable IDs.

Encoding & scaling

Encode categorical variables; scale numeric features when the algorithm needs it.

Leakage control

Exclude fields that reveal the landing result; split train/test before fitting preprocessing.

Reproducibility

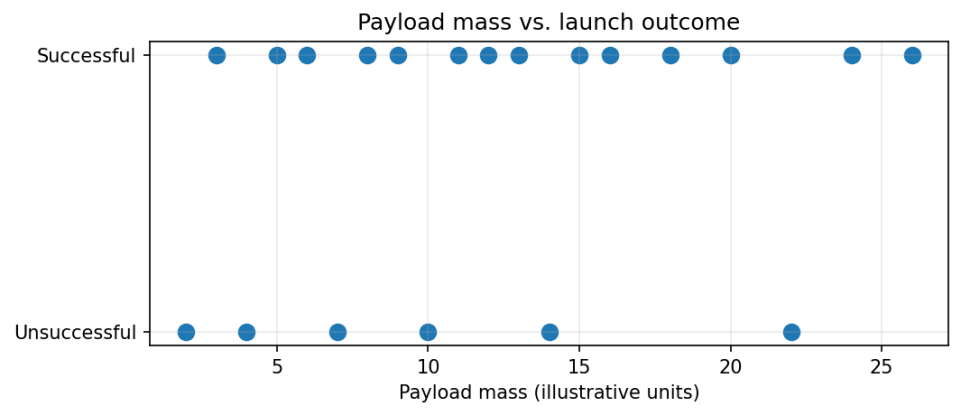
Keep transformations in a pipeline and set random_state for repeatable splits.

GitHub URL: ADD YOUR REPOSITORY URL HERE

EDA with Data Visualization

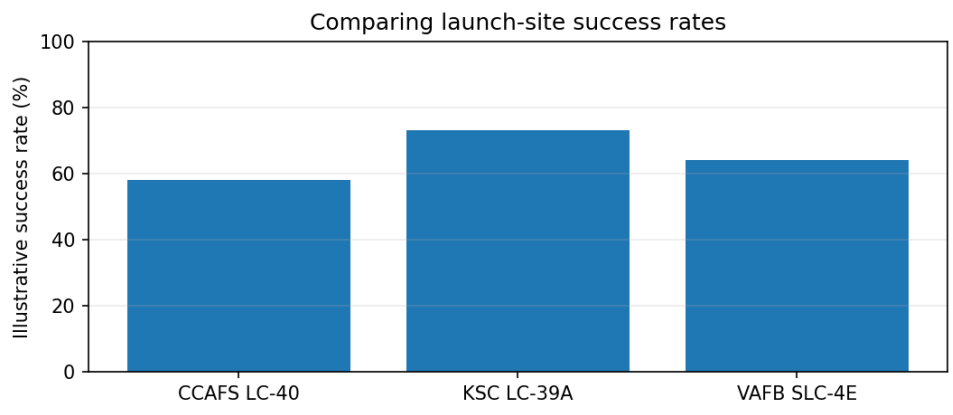
Scatter plot

Explore whether payload mass differs by landing outcome.



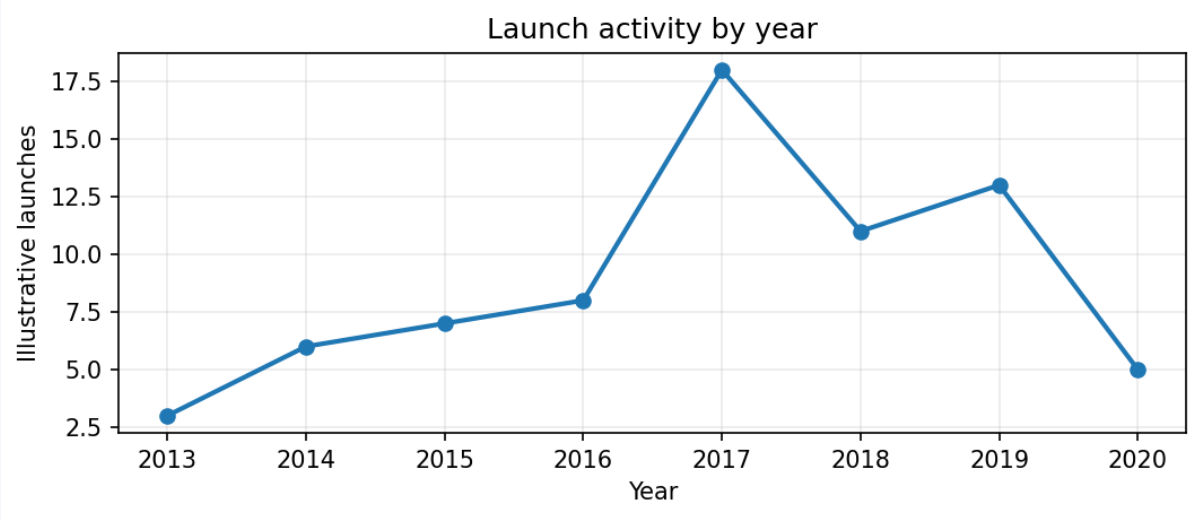
Bar chart

Compare outcome rates across launch sites.



Charts are layout examples with illustrative values. Replace with charts from your analysis.

EDA Results — Launch Activity Over Time



Questions to answer

Did launch frequency increase over time?
Do success rates change by year?
Are there years with small sample sizes?

Interpret carefully

Separate launch volume from success rate;
show sample counts and denominators.

Illustrative trend only — regenerate using the actual cleaned dataset.

EDA with SQL — Query Plan

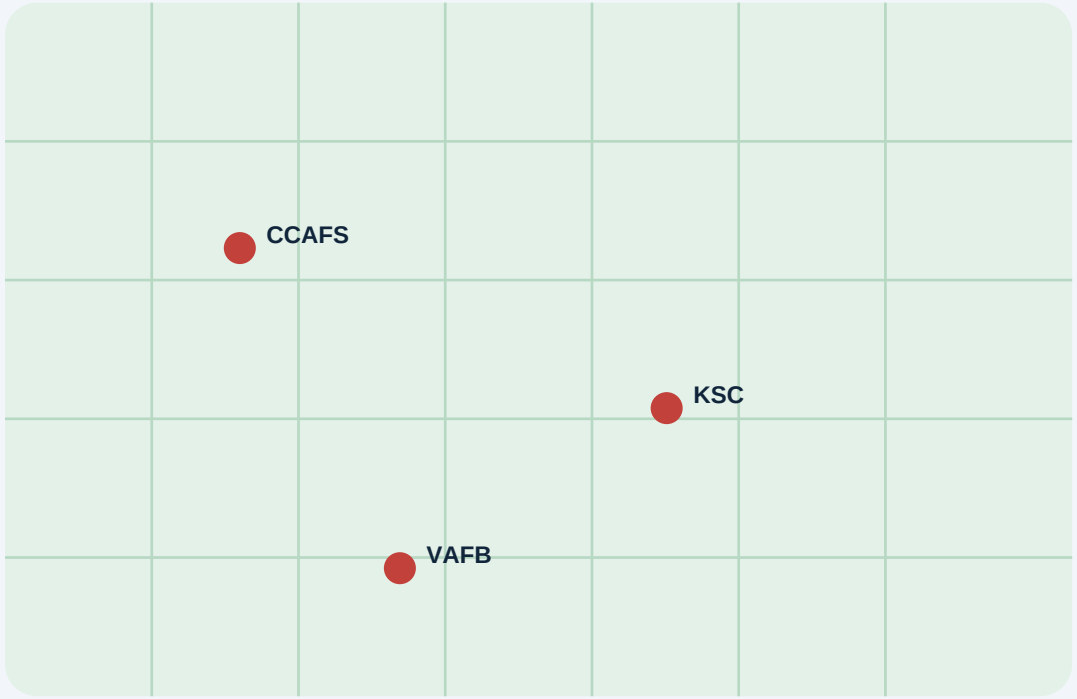
Launch sites	Count launches and successful outcomes by launch site.
Payloads	Summarize payload mass by orbit, customer/mission class and outcome.
Success rates	Calculate successes / total launches; handle zero denominators.
Rankings	Rank sites and orbit categories with minimum sample-size checks.
Time analysis	Group launches by year/month and compare counts with success rates.

Include the executed SQL cells/results in the GitHub notebook. Repository URL: ADD HERE

Interactive Visual Analytics — Folium Map

Map layers

- Launch-site markers
- Popups with launch records
- Success/failure attributes
- Nearby geography / proximity



Schematic only, not a geographic map. Replace with a Folium screenshot from your notebook.

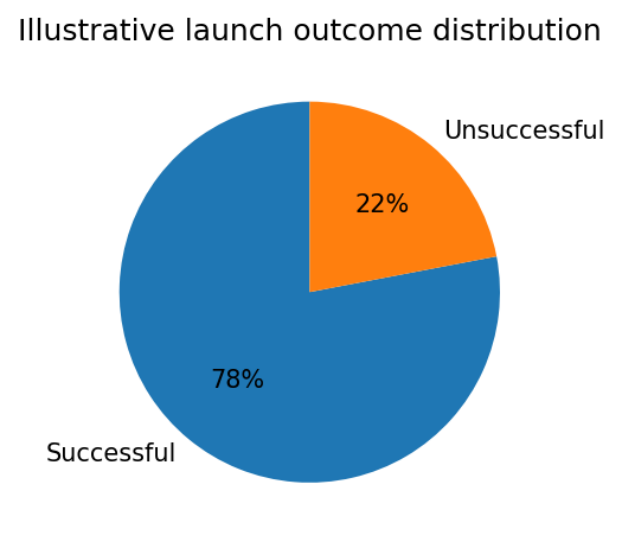
Proximity analysis

Use geodesic distance or a distance matrix to identify nearby sites/infrastructure.

Interactive Visual Analytics — Plotly Dash

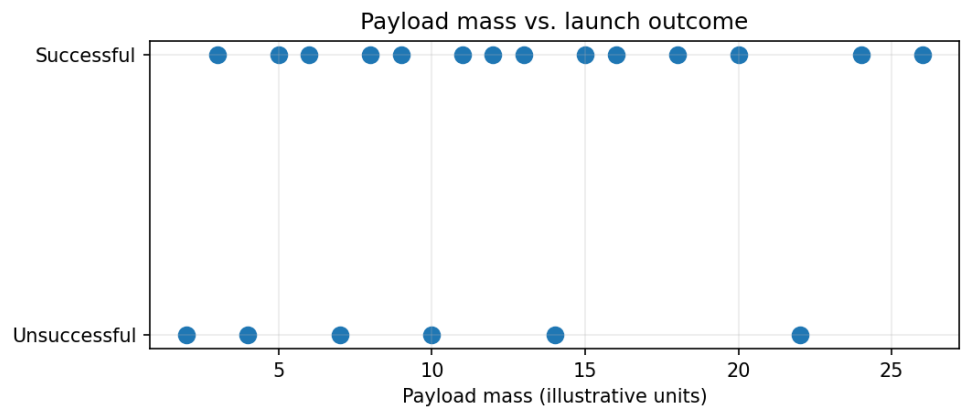
Success pie chart

Shows proportion of successful and unsuccessful outcomes.



Payload vs. outcome

Explore payload distribution by launch outcome.



Illustrative chart data. Add a real Dash app screenshot and repository link.

EDA with SQL — Required Result Slides

Launch-site summary

Show launch count, success count and success percentage by site.

Payload analysis

Show payload distribution and outcomes by mass range / orbit.

Ranking & time

Rank sites/orbits; plot yearly/monthly volume and success rate.

SQL result table template

Dimension	Metric(s) to display	Interpretation
Launch site	Total, successes, rate	Compare sites fairly
Payload / orbit	Count, median mass, rate	Identify differences
Rankings	Rank + sample count	Avoid tiny-sample bias
Time	Launches/year, rate/year	Trend vs. volume

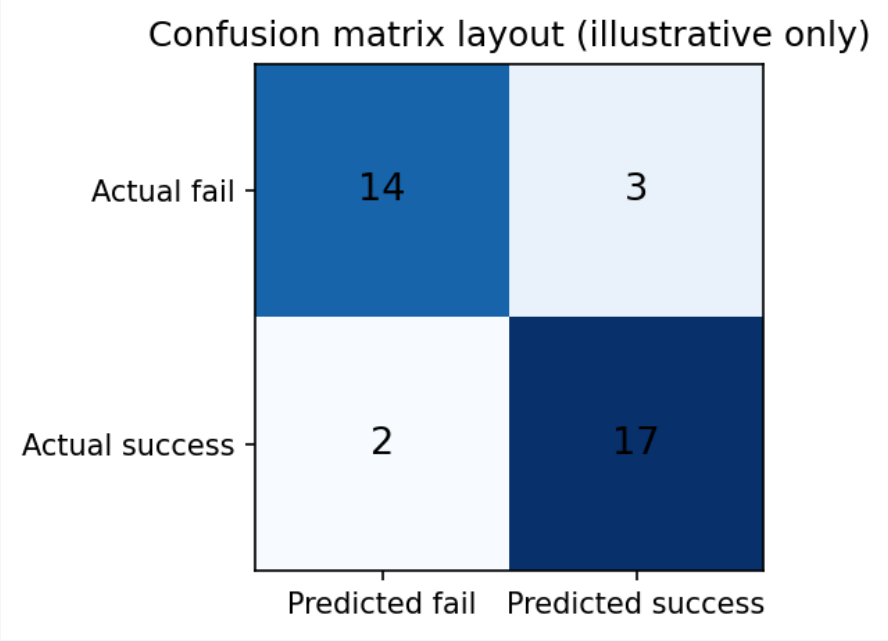
Populate these summaries from executed SQL queries; do not submit blank or invented results.

Predictive Analysis — Modeling Strategy

Logistic Regression	Interpretable baseline; coefficient direction can aid explanation.
Support Vector Machine	Tests a nonlinear / margin-based decision boundary.
Decision Tree	Readable rules; useful for comparing feature splits.
K-Nearest Neighbors	Distance-based baseline; scale numeric features.
Model selection	Use stratified split / cross-validation and tune on training data only.

Report test accuracy plus precision, recall, F1 and confusion matrix; include class balance.

Predictive Analysis — Evaluation & Confusion Matrix



Metrics to report

- Accuracy: overall fraction classified correctly
- Precision: how often predicted successes are correct
- Recall: fraction of actual successes detected
- F1-score: balance of precision and recall
- Confusion matrix: inspect false positives and false negatives

Illustrative matrix only — replace with actual
test-set predictions from the best model.

Best Model — How to Justify the Choice

1. Compare fairly

Same held-out test set and target definition for all candidate models.

2. Balance metrics

Consider F1 / recall as well as accuracy, especially if classes are imbalanced.

3. Explain trade-offs

Interpretability, stability and error types matter alongside headline performance.

Final result to insert after running the notebook

Best model: [insert model name from evaluation]

Test accuracy / precision / recall / F1: [insert measured values]

Do not choose a model or claim a score until the comparison has actually been run.

Conclusion & Actionable Insights

- Integrating API records and scraped launch data creates a richer analytical dataset.
- EDA and SQL can reveal differences in launch volume, payload mix, orbit and launch-site outcomes.
- Folium communicates geographic context; Plotly Dash supports interactive exploration.
- Predictive models estimate historical outcome patterns but should not be treated as operational launch decisions.
- Next steps: validate against newer records, monitor drift, document limitations and reproduce results from the repository.

Creative insight

Combine success probability with payload and site context to create an exploratory mission-planning view.

Responsible use

Report uncertainty, class balance, sample size and the cost of each type of prediction error.

Submission Checklist — Finish Before Upload

- Create a public GitHub repository for the project.
- Upload notebook(s), data-processing code, SQL queries and README.
- Run all cells; replace every illustrative chart with real outputs.
- Add actual Folium map and Plotly Dash screenshots.
- Insert repository URL on required slides.
- Replace model name and evaluation metrics with measured values.
- Export this slide deck to PDF and confirm all text is readable.

GitHub repository URL

PASTE URL HERE AFTER CREATING THE REPOSITORY

The repository URL cannot be generated without your GitHub account/repository.